```
// 🧩 Product Analyzer with Higher-Order Functions
// ----------------------------------------------
// In this activity, you'll create a mini Product Analyzer that uses classes,
// collections, and higher-order functions to process product data efficiently.
//
// Your program should store products, calculate discounts, and apply filters dynamically.
// Practice working with functions as parameters and Kotlin collections.
//
// 💡 You'll learn how to:
// 1. Create and use classes
// 2. Work with lists and higher-order functions
// 3. Use lambdas to filter and transform data


// -------------------------------
// TODO 1: Define a Product class with:
// - name (String)
// - category (String)
// - price (Double)
// - stock (Int)
// -------------------------------



// -------------------------------
// TODO 2: Create a higher-order function called `applyDiscount`
// that takes:
// - a list of products
// - a discount function: (Double) -> Double
```

// It should return a **new list** of products with the discounted price applied.

// Hint: Use `map()` to transform the list.

// ---------------------------------

// ---------------------------------

// TODO 3: Create a higher-order function called `filterProducts`

// that takes:

// - a list of products

// - a condition function: (Product) -> Boolean

// It should return a **filtered list** based on the condition.

// ---------------------------------

// ---------------------------------

// TODO 4: In `main()`:

// 1. Create a sample product list (at least 5 products).

// 2. Use `applyDiscount` to reduce all prices by 10%.

// 3. Use `filterProducts` to find:

// a) Products with stock less than 10

// b) Products with price greater than 1000

// 4. Print results using clear, formatted output.

// Hint: Use string templates and loops.

// ---------------------------------

// ---------------------------------

// 📜 SAMPLE OUTPUT (for reference)

```
// -------------------------------
/*
--- Discounted Product List ---
Name: Laptop | Category: Electronics | Price: 90000.0 | Stock: 5
Name: Phone | Category: Electronics | Price: 45000.0 | Stock: 15
Name: Chair | Category: Furniture | Price: 1800.0 | Stock: 25
Name: Table | Category: Furniture | Price: 2700.0 | Stock: 5
Name: Pen | Category: Stationery | Price: 45.0 | Stock: 100

--- Low Stock Products (<10) ---
Laptop (5)
Table (5)

--- Premium Products (>1000) ---
Laptop (90000.0)
Phone (45000.0)
Chair (1800.0)
Table (2700.0)
*/
// -------------------------------
fun main() {
   // TODO 5: Implement your logic here
}
```

```
data class Product(
    val name: String,
    val category: String,
    val price: Double,
    val stock: Int
)


fun applyDiscount(
    products: List<Product>,
    discount: (Double) -> Double
): List<Product> {
    return products.map { it.copy(price = discount(it.price)) }
}


fun filterProducts(
    products: List<Product>,
    condition: (Product) -> Boolean
): List<Product> {
    return products.filter(condition)
}


fun main() {
```

```
    val productList = listOf(
        Product("Laptop", "Electronics", 100000.0, 5),
        Product("Phone", "Electronics", 50000.0, 15),
        Product("Chair", "Furniture", 2000.0, 25),
        Product("Table", "Furniture", 3000.0, 5),
        Product("Pen", "Stationery", 50.0, 100)
    )

    t
    val discountedProducts = applyDiscount(productList) { price ->
        price * 0.9
    }

    println("--- Discounted Product List ---")
    for (product in discountedProducts) {
        println("Name: ${product.name} | Category: ${product.category} | Price:
${product.price} | Stock: ${product.stock}")
    }



    val lowStockProducts = filterProducts(productList) { product ->
        product.stock < 10
    }

    println("\n--- Low Stock Products (<10) ---")
    for (product in lowStockProducts) {
        println("${product.name} (${product.stock})")
```

```
    }



    val premiumProducts = filterProducts(productList) { product ->
        product.price > 1000
    }

    println("\n--- Premium Products (>1000) ---")
    for (product in premiumProducts) {
        println("${product.name} (${product.price})")
    }
}
```